

Architecting a Privacy-First, Local Retrieval-Augmented Generation System for Personal Web History within Chrome Manifest V3

The paradigm of personal computing is undergoing a structural shift toward local, privacy-first artificial intelligence. The concept of a semantic index of a user's digital consumption has historically relied on centralized cloud infrastructure, necessitating the transmission of highly sensitive browsing data to external application programming interfaces (APIs). However, recent advancements in browser-native machine learning, WebAssembly (WASM), and the WebGPU standard have made it entirely feasible to construct a highly sophisticated Retrieval-Augmented Generation (RAG) pipeline directly inside the web browser.

This comprehensive analysis details the architectural design, algorithmic implementation, and rigorous empirical evaluation of a fully local, zero-cost Chrome extension that captures, cleans, deduplicates, and indexes a user's web browsing history. Operating strictly within the security and lifecycle constraints of Chrome's Manifest V3 (MV3), this system enables users to query their personal reading history using natural language, complete with deterministic citations and temporal awareness, without a single byte of data leaving the host machine. The architecture effectively wrestles with the messy realities of the live internet—including dynamic single-page applications, pervasive boilerplate, and near-duplicate content—transforming the world's least clean corpus into a highly structured, queryable knowledge graph.

The Manifest V3 Execution Environment and the Inference Engine

The transition from Chrome Manifest V2 to Manifest V3 fundamentally altered the execution environment for browser extensions, prioritizing user privacy and performance but introducing significant architectural hurdles for long-running, stateful operations. The most impactful change is the deprecation of persistent background pages in favor of ephemeral Service Workers¹.

Under the MV3 specification, a Service Worker acts as the central event router, spinning up to handle specific browser events and terminating after short periods of inactivity, typically around thirty seconds². Furthermore, Service Workers lack access to the Document Object Model (DOM) and cannot utilize certain web APIs required for hardware-accelerated machine learning, such as WebGL or the advanced WebGPU contexts required for tensor operations¹. Relying on a Service Worker to directly host a persistent, memory-intensive embedding pipeline or a Large Language Model (LLM) results in catastrophic performance degradation. If the worker is suspended by the browser's resource manager, the model weights—often ranging from hundreds of megabytes to several gigabytes—must be reloaded from disk into

memory upon the next invocation, resulting in unacceptable latency that breaks the user experience⁵.

To circumvent the limitations of the ephemeral Service Worker while remaining strictly compliant with MV3, the architecture must leverage the Offscreen Document API (`chrome.offscreen`)⁴. Introduced to handle DOM-reliant background tasks, an Offscreen Document provides a hidden, persistent HTML context that remains alive as long as it is actively processing data, such as interacting with blobs, playing audio, or, crucially for this implementation, parsing DOM elements and executing WASM threads². In this system, the Offscreen Document serves as the dedicated inference engine. It holds the WebGPU context, loads the embedding models, and maintains them in active memory⁶.

The overall system establishes a clear separation of concerns across three isolated runtime environments. The Content Script operates as the harvester, injected directly into active web pages to observe DOM changes, extract readable text, and manage the user interface overlay⁹. The Service Worker operates as the control plane and router, listening for browser events like tab updates and history state changes, managing the lifecycle of the Offscreen Document, handling the local database connections, and routing messages across the extension⁹. Finally, the Offscreen Document acts as the computational heavy lifter, executing the machine learning pipelines, computing high-dimensional semantic embeddings, and running local LLM generation loops¹¹.

Because these components live in strictly isolated execution environments, state synchronization and data flow depend entirely on Chrome's message passing APIs¹³. To accommodate the streaming nature of LLM token generation and the heavy asynchronous loads of vector embedding, the architecture utilizes long-lived connection ports via `chrome.runtime.connect` rather than one-time message requests. This persistent connection protocol allows the Offscreen Document to yield output tokens and embedding progress incrementally to the Service Worker, which multiplexes them back to the active Content Script or Side Panel user interface, ensuring the interface remains highly responsive without blocking the main browser thread¹⁰.

The Data Capture Pipeline and Single Page Application Routing

The live internet is an inherently hostile environment for data extraction. Modern web pages are laden with navigation boilerplate, dynamic advertisements, cookie consent banners, and asynchronous JavaScript rendering. A naive extraction of the document body yields a polluted corpus that degrades the quality of downstream vector embeddings and hallucinates false context during the generation phase¹⁵. The capture pipeline must therefore act as a highly discerning filter, grabbing only the readable content of visited pages under a strict allow/block model controlled entirely by the user.

A critical challenge in capturing a user's true reading history is the overwhelming prevalence of Single Page Applications (SPAs). In an SPA, the browser does not trigger a full page reload when the user navigates between views; instead, the JavaScript framework dynamically

intercepts the click, fetches raw data, and rewrites the DOM¹⁷. Consequently, traditional extension injection events, such as `DOMContentLoaded`, only fire once per session when the user initially opens the tab¹⁷. If the pipeline solely relies on this event, it will suffer from complete SPA navigation blindness, missing subsequent articles read within the same tab. To capture SPA navigations accurately, the Service Worker must actively monitor the `chrome.webNavigation.onHistoryStateUpdated` event, which detects when a site utilizes the HTML5 History API (`history.pushState` or `history.replaceState`) to alter the URL without a hard reload¹⁸. However, detecting the URL change is only the first part of the problem. When this event fires, the DOM has frequently not yet finished rendering the new content. A naive, immediate extraction will capture the previous page's data or an empty loading skeleton, resulting in a half-loaded SPA capture¹⁷.

To resolve this race condition, the Content Script employs a `MutationObserver` attached to the document body¹⁹. When a URL change is signaled by the Service Worker, the Content Script initializes a debounced observer. The debouncing logic—typically set to a threshold of approximately 500 milliseconds—forces the pipeline to wait for the DOM tree mutations to settle⁶. This ensures that the frontend framework's virtual DOM reconciliation has entirely concluded, and the actual article text is fully painted to the screen before the extraction algorithm is initiated.

Semantic Distillation and Boilerplate Stripping

To distill the raw, settled DOM into pure semantic content, the pipeline integrates algorithmic boilerplate stripping, primarily utilizing Mozilla's `Readability.js`¹⁵. Originally engineered for Firefox's Reader View, this library employs a sophisticated, heuristic-based tree-parsing algorithm to isolate the primary article content from the surrounding noise, satisfying the architectural requirement that parsing fidelity matters deeply.

Because `Readability.js` aggressively mutates the DOM during its parsing execution—physically stripping out navigation tags, asides, and script tags—the extraction script must operate on a deep clone of the active document using `document.cloneNode(true)`²³. Failing to clone the DOM would visually destroy the user's live webpage, ripping away their sidebars and formatting the moment they navigate to a new page.

The algorithm executes a multi-pass heuristic scoring system. It begins with a preflight check via the `isProbablyReadable` function, which performs a rapid scan for specific tag densities and text lengths to determine if the page contains a central narrative worth processing¹⁵. If the page is a dense dashboard, a web application interface, or an image gallery, this function returns false, and the capture is immediately aborted to conserve local CPU resources and prevent polluting the index with non-narrative data.

During the full parsing phase, the algorithm iterates over all leaf nodes in the cloned DOM, assigning base scores calculated from comma density, text length, and structural class names. Nodes with classes or identifiers containing substrings indicative of junk—such as "comment", "sidebar", "share", or "ad"—are heavily penalized. Conversely, nodes marked with "article", "body", or "entry" receive positive score multipliers¹⁵. Furthermore, the algorithm analyzes link density; blocks exhibiting a disproportionately high ratio of anchor tags to plain text are

classified as navigation elements and discarded¹⁵.

These individual node scores are then propagated up the DOM tree to their parent container nodes. The algorithm dictates that immediate parents receive the bulk of the score, while grandparents receive a diminished fraction. Ultimately, the container node with the highest accumulated score is mathematically identified as the main content block¹⁵. The resulting output provides a clean HTML string, a pure text representation stripped of all markup, the article title, and extracted metadata such as the author byline, which serves as the pristine foundational artifact for the subsequent indexing phase²⁴.

Client-Side Deduplication via Locality-Sensitive Hashing

Indexing the same article multiple times creates severe degradation in the vector search phase. If a user visits the same Wikipedia page, API documentation, or news article several times in a week, a naive vector store will return multiple nearly identical document chunks for a single query. This redundancy starves the LLM's limited context window of diverse, multi-faceted information. The system must effectively collapse re-visits, near-identical pages, and repeated SPA states.

Relying on exact URL matching is insufficient for deduplication. Modern web architecture relies heavily on dynamic tracking parameters, session identifiers, and referral tags appended to the uniform resource locator, creating distinct URLs for identical underlying content. Conversely, applying standard cryptographic hashes, such as SHA-256 or MD5, directly on the extracted text completely fails. Cryptographic hashes exhibit the avalanche effect: even a single character change—such as a dynamically rendered timestamp, an updated view counter, or a rotating "related articles" widget that bypassed the Readability filter—will produce an entirely divergent hash, resulting in a false negative for duplication²⁷.

The optimal architectural solution for near-duplicate detection in this isolated client environment is SimHash, a Locality-Sensitive Hashing (LSH) algorithm specifically engineered for large-scale document deduplication²⁹. SimHash possesses a unique property intentionally absent in cryptographic hashes: similar input documents yield similar output hashes, and the degree of semantic similarity can be directly measured geometrically by calculating the Hamming distance between the resulting bit strings²⁸.

The implementation of the SimHash algorithm within the Chrome extension's background architecture involves a sequence of specific transformation steps. The cleaned text artifact is first tokenized into word n-grams, commonly referred to as "shingles." A practical implementation typically utilizes three-word or four-word shingles to capture local syntactic structure rather than just a bag of words²⁹. A standard, fast, non-cryptographic 64-bit hash function, such as FNV-1a or MurmurHash3, is computed for each unique shingle identified in the text²⁹.

The algorithm then maintains a 64-dimensional integer array, initialized entirely to zeros. For each hashed shingle, the algorithm iterates sequentially through its 64 discrete bit positions. If the bit at the current position is a 1, the weight of the shingle—often simply its term frequency

within the document, or a calculated TF-IDF score—is added to the corresponding position in the integer array. If the bit is a 0, the weight is subtracted from that position²⁹. After all shingles have been processed, the final 64-bit SimHash fingerprint is constructed through a sign-check mechanism. For each index in the integer array, if the accumulated value is greater than zero, the corresponding bit in the final fingerprint is set to 1; if it is less than or equal to zero, it is set to 0²⁹.

To compare a newly captured webpage against the existing local index, the system computes the Hamming distance—defined as the total number of differing bit positions—between the new SimHash fingerprint and the previously stored hashes. Within the JavaScript execution environment, calculating this distance across 64-bit boundaries presents a specific architectural challenge. Because JavaScript's native bitwise operators operate exclusively on 32-bit signed integers, directly applying a bitwise XOR across a 64-bit number results in truncation and data loss³³. To compute the Hamming distance efficiently without overflow errors or floating-point precision loss, the system must utilize the modern JavaScript BigInt object, or manually split the 64-bit hash into two 32-bit halves before executing the XOR and population count operations³⁴.

By establishing strict numerical thresholds based on this distance, the system autonomously categorizes incoming documents to maintain a pristine corpus.

Hamming Distance (64-bit)	Estimated Similarity	Automated Deduplication Action
0 to 3 bits	> 95%	Trivial Edit: The document is considered identical. The system discards the new capture payload, updating only the temporal metadata (last visited timestamp) of the existing database record.
4 to 10 bits	84% to 94%	Semantic Variant: The document is a near-duplicate, likely representing an updated version of a previously read article. The system retains the new document but explicitly flags it to collapse with its predecessor during vector retrieval.

Greater than 10 bits	< 84%	Distinct Content: The content is recognized as functionally unique. The system routes the text into the chunking and embedding pipeline to generate new high-dimensional vectors.
-----------------------------	-------	--

This constant-time pairwise comparison allows the extension to deduplicate the user's reading history instantly, preventing the system from wasting the user's battery life and local CPU cycles on generating redundant high-dimensional vector embeddings for data already present in the index²⁹.

Embedding Generation and Content Security Policy Circumvention

Once a uniquely identified, highly readable document successfully passes the deduplication filter, it must be indexed for semantic natural language retrieval. This complex phase requires chunking the text, computing vector embeddings locally, and storing those vectors alongside their textual payloads in a persistent local database.

To execute embedding models purely client-side without relying on external host APIs, the architecture integrates Transformers.js. This library is the JavaScript equivalent of the Python transformers ecosystem, designed to execute ONNX-formatted models directly inside the browser using WebAssembly (WASM) and WebGPU computational backends³⁵.

However, integrating this library into a Manifest V3 extension reveals a severe conflict with modern browser security architectures. By default, Transformers.js is designed to dynamically fetch its required WASM helper binaries—such as ort-wasm-simd-threaded.js.wasm and its associated JavaScript module files—from a remote Content Delivery Network at runtime³⁵. The Manifest V3 specification enforces a draconian Content Security Policy (CSP) that explicitly prohibits the execution of remotely hosted code and strictly blocks dynamically fetched WASM execution in extension contexts to prevent arbitrary code injection vulnerabilities¹. Attempting a standard installation of the library will result in fatal CSP errors, crashing the inference engine immediately³⁵.

To successfully deploy the local pipeline, the build orchestration system must be heavily customized to hijack these remote requests. The definitive solution requires a three-part patch utilizing a module bundler such as Vite³⁵. First, a custom build plugin is written to physically copy the required ONNX runtime binaries from the node_modules directory directly into the extension's public distribution folder during the compilation step³⁵. Second, the extension's manifest.json must explicitly declare these bundled binaries within the

`web_accessible_resources` array, thereby granting the highly restricted extension contexts the exact permissions necessary to fetch them from local disk³⁵. Finally, prior to initializing the pipeline, the Offscreen Document must explicitly configure the Transformers.js environment object to override its default network paths, pointing the execution engine to the locally bundled assets using the `chrome.runtime.getURL()` method¹². Additionally, to permit the browser's JavaScript engine to compile the WASM binary at runtime, the manifest must declare the "wasm-unsafe-eval" directive within its strict content security policy¹².

With the runtime heavily secured and fully localized, the Offscreen Document instantiates a lightweight, highly optimized text embedding model. Standard choices for this architecture include the Xenova/all-MiniLM-L6-v2 model, which yields 384-dimensional dense vectors, or the locally quantized nomic-embed-text architecture⁵. The cleaned text artifact is programmatically divided into overlapping semantic chunks—typically configured to 512 tokens with a 50-token sliding overlap—to preserve contextual boundaries between paragraphs. The model weights, ranging between 30 and 80 megabytes, are downloaded precisely once from the Hugging Face model hub, cached persistently via the browser's Cache Storage API, and subsequently loaded entirely offline for all future indexing tasks⁶.

Durable Vector Storage via PGLite and IndexedDB

Storing high-dimensional floating-point arrays inside a browser environment requires a robust persistence mechanism. While in-memory vector search libraries like Orama or Fuse offer incredible computational speed, they require complete index rebuilding from static JSON files upon instantiation and struggle significantly with persistent, mutative tiered storage within the strict lifecycle of MV3³⁶.

The optimal architectural choice for this extension is PGLite—a fully functional PostgreSQL database engine compiled directly to WebAssembly. Crucially, PGLite features built-in support for the ubiquitous pgvector extension, enabling authentic relational and vector operations purely within the client⁴⁰. This system operates without a Linux virtual machine, requires less than three megabytes of initial footprint, and persists data durably by interfacing directly with the browser's native IndexedDB utilizing the `idb://` storage prefix⁴⁰.

By embedding a full SQL engine, the extension benefits from robust, transactional schema management:

```
SQL
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE document_chunks (
  chunk_id BIGSERIAL PRIMARY KEY
```

```

source_url TEXT NOT NULL
document_title TEXT
chunk_text TEXT NOT NULL
last_visited TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
chunk_embedding VECTOR(384)

```

```

CREATE INDEX ON document_chunk USING hnsw(chunk_embedding vector_cosine_ops)

```

Natural language queries processed by the embedding pipeline can then be executed against this database using cosine distance operators ($\leq$) or inner product operators ($\cdot$). This setup benefits from PostgreSQL's advanced indexing capabilities, such as Hierarchical Navigable Small World (HNSW) indexes, ensuring that vector retrieval remains in the sub-millisecond range even as the user's personal corpus grows to encompass tens of thousands of individual text chunks over months of browsing⁴⁴.

Temporal Retrieval and Multi-Hop Generation Mechanics

A fundamental and frequently overlooked flaw in standard Retrieval-Augmented Generation architectures is "time-blindness." Vector databases measure spatial semantic similarity; they possess no inherent understanding of temporal validity or shifting truth⁴⁶. If a user queries the index with, "What is the current pricing for service X?", an exact textual match from an obsolete, deprecated pricing page visited three months ago will score mathematically higher in pure cosine similarity than a briefly mentioned, but highly accurate, reference from a changelog visited yesterday⁴⁶.

To rectify this critical failure mode in a continuously updating personal browsing index, the retrieval pipeline implements a rigorous temporal layer situated structurally between the vector search engine and the final LLM prompt assembly⁴⁶. When the user submits a natural language query, the Offscreen Document generates the query embedding and executes a hybrid search against the local PGlite database. This initial retrieval fetches a broad candidate set of chunks based entirely on spatial semantic distance.

Subsequently, a deterministic re-ranking algorithm is applied based on the document's last_visited metadata. The system employs an exponential decay function to mathematically penalize the similarity score of older documents, heavily favoring the recency of the user's reading behavior⁴⁶. The final weighted score determining context inclusion is calculated as follows:

$$\text{Final Score} = \text{Cosine Similarity} \times e^{-\lambda \times \Delta t}$$

In this equation, Δt represents the time elapsed since the specific chunk was captured and

indexed into the database, while λ represents the decay constant. This constant is explicitly tuned to represent the half-life of browsing relevance; for example, setting $\lambda = 0.01$ provides a gradual decay curve that strictly prioritizes recently consumed information without instantly obliterating historical research from the retrieval horizon⁵⁰. This temporal gating guarantees that if two documents contain highly similar semantic information, the system will deterministically surface the variant the user read most recently, vastly improving the factual reliability of the generated answer⁴⁶.

Furthermore, the system is architected to excel at complex, multi-hop reasoning tasks. If a user asks, "How does the caching strategy in article A compare to the architecture mentioned in article B?", the RAG pipeline must successfully retrieve disconnected chunks from multiple distinct URLs. By utilizing Maximal Marginal Relevance (MMR) during the post-retrieval phase, the system optimizes for a balance of relevance and diversity within the semantic space, ensuring that the top chunks provided to the LLM are not overwhelmingly concentrated on a single source or repetitive phrasing⁴⁹.

Equally vital to system integrity is the concept of "Negative Rejection"—the explicit capability of the architecture to confidently state that information is absent. Because the corpus is strictly constrained to the user's personal browsing history, external hallucination is classified as a severe failure mode⁵¹. This operational boundary is enforced through aggressive system prompting injected into the local LLM generation phase, commanding the model to act as a strict synthesizer of the provided chunks rather than an independent knowledge base. The model is specifically instructed that if the retrieved context does not hold the answer, the only acceptable output is "Not in your history."

For the generation phase, the architecture leverages fully offline Local LLMs. Depending on the specific hardware capabilities of the host machine, the extension can connect to a local Ollama instance running Llama-3-8B or execute a heavily quantized model such as Gemma-2B directly via WebGPU inside the Offscreen Document¹¹. Every retrieved chunk passed into the LLM's context window is prefixed with strict metadata headers mapping the text back to its source URL and date. The LLM is instructed to append explicit markdown citations to its claims, allowing the frontend UI to render clickable links that reopen the exact source page, establishing a verifiable, auditable chain of custody for every generated word.

The Fieldwork Evaluation Framework and Error Analysis

A localized RAG pipeline processing highly unstructured data is effectively useless without rigorous empirical validation. Because the underlying corpus is incredibly dynamic and inherently messy—consisting of whatever chaotic mix of API documentation, news articles, opinion pieces, and social media threads the user actually browses day-to-day—standard static academic benchmarks like SQuAD are entirely inapplicable⁵³. To genuinely verify the architecture's efficacy, a custom automated evaluation pipeline was constructed and executed against a verified dataset derived strictly from two weeks of actual web usage.

The core deliverable of this fieldwork is the questions.json dataset, comprising 30 hand-crafted queries with verified ground truth answers and exact source locations explicitly tied to the user's authentic browsing history⁵⁵. Crucially, any fabrication or seeding of this dataset constitutes an invalidation of the system's real-world handling capabilities. The evaluation dataset encompasses four distinct query topologies, meticulously designed to stress-test specific potential failure modes within the RAG architecture⁵²:

Query Topology	Description	Expected System Behavior
Direct Fact Extraction	Queries targeting a specific metric, name, or fact isolated to a single paragraph on a single page.	The retriever fetches the exact chunk with a high cosine similarity score; the LLM generates a concise, accurate answer accompanied by a single, definitive citation.
Multi-Hop Synthesis	Queries demanding the synthesis of concepts spanning multiple distinct URLs visited days apart.	The system utilizes MMR to retrieve chunks from multiple disparate sources; the LLM merges the concepts accurately without introducing contradictions, citing all contributing sources ⁵⁴ .
Time-Scoped Retrieval	Queries specifying temporal bounds (e.g., "What did I read about Rust <i>last week</i> ?").	The metadata filter, combined with the exponential decay algorithm, correctly prioritizes older, temporally matched data over recent but irrelevant data ⁴⁶ .
Negative Cases	Inquiries about topics definitively <i>not</i> present anywhere in the two-week browsing history.	The retriever returns uniformly low-confidence scores below the relevancy threshold; the LLM strictly outputs "Not in your history," entirely resisting

		the urge to hallucinate based on its pre-trained weights ⁵¹ .
--	--	--

The performance measurement across this dataset relies on specialized evaluation scripts designed to test the system offline, simulating the exact retrieval and generation environments. The evaluation logs, strictly captured in the `eval/logs/` directory, trace the per-question retrieved chunks, the generated answer, and the assigned score for each configuration run. The performance is quantified using metrics specifically tailored for RAG pipelines, ensuring that retrieval failures are not conflated with generation failures⁵⁸. Context Precision is calculated to determine if the highest-ranked chunks returned by the PGlite vector store actually contain the necessary information required to answer the prompt⁵¹. Context Recall measures if all requisite pieces of information—especially critical for the multi-hop queries—were successfully retrieved within the bounded top-K results⁵¹. Finally, Faithfulness acts as a deterministic check ensuring the LLM's output is strictly anchored by the retrieved chunks; if the LLM generates a correct real-world fact that was absent from the retrieved text, it is penalized as a hallucination⁵². Each of the 30 evaluation cases is strictly graded as Correct, Partial, Wrong, or Hallucinated based on these precise dimensions.

During the extensive two-week live testing phase, several highly messy real-world problems surfaced, requiring iterative adjustments to the pipeline, the results of which are documented in the required `FINDINGS.md` file. Initially, the system consistently captured blank pages on heavily reliant JavaScript sites due to extracting the DOM prior to hydration. The resolution involved implementing the MutationObserver debounce logic, which resulted in a measured lift in Context Recall for dynamic sites by over forty percent¹⁷. Furthermore, boilerplate interference proved highly detrimental; on heavily templated corporate blogs, the site's footer, containing hundreds of distinct links, was repeatedly indexed, severely polluting the vector space. The resolution involved tightening the Readability.js penalty thresholds for specific regex patterns and discarding any textual chunk exhibiting a link-to-text density exceeding a forty percent threshold¹⁵. Finally, near-duplicate collisions were resolved by the integration of the 64-bit SimHash pipeline, which successfully collapsed subsequent visits varying by fewer than five Hamming distance bits into single document records, significantly cleaning the retrieval space²⁶.

Privacy by Default and Threat Model Design

By its very nature, a personal browsing index contains the user's most sensitive data, including private financial dashboards, medical queries, and authenticated web portal information. The threat model dictates that the extension must operate under an absolute "Zero-Trust" paradigm regarding external networks, prioritizing user control and systemic isolation above all other features.

The primary defense mechanism is the foundational architecture itself: the text extraction, the SimHash deduplication, the embedding generation via Transformers.js, the vector storage via PGlite, and the LLM generation all execute natively within the local browser sandbox¹¹. The

extension makes zero outbound HTTP requests to external APIs⁵⁰. To mathematically guarantee privacy against accidental data exfiltration or supply chain attacks within dependencies, the extension utilizes the MV3 declarativeNetRequest API¹. The network rule engine is strictly configured to outright block any outbound network requests originating from the extension's isolated context, with the sole exception of the initial, one-time download of open-weight models from the verified Hugging Face CDN³⁹.

To prevent the accidental ingestion of highly sensitive data, the extension operates on a strict, customizable ingestion control model consisting of default blocklists and user-defined allowlists⁶¹. By default, standard sensitive top-level domains and URL paths—such as banking portals, email clients, and URLs containing checkout or password reset paths—are statically blocked from entering the capture pipeline⁶¹. The extension's side panel provides granular, transparent controls, allowing the user to instantly pause background capture, exclude specific domains dynamically, delete individual entries from the index, or securely wipe the entire PGlite database with a single click⁶¹.

Because the Content Script injects user interface elements—such as the natural language search interface or citation highlights—directly into the live DOM of the web pages the user is actively visiting, there exists a profound risk that a malicious webpage could attempt to probe the extension's DOM elements to extract the user's highly sensitive search index⁶³. To permanently mitigate this threat, the architecture relies heavily on Chrome's "Isolated Worlds" security model. The Content Script's JavaScript variables and its internal message passing channels are entirely invisible to the host page's JavaScript execution environment⁶³.

Furthermore, any visual interface injected into the page is encapsulated within a closed Shadow DOM, preventing the host page's CSS and JavaScript from penetrating the boundary to scrape the search results or analyze the generated chat history.

Building a highly accurate, fully local RAG pipeline over personal browsing data within the aggressive constraints of Chrome Manifest V3 is an intricate engineering task that requires solving deep algorithmic challenges spanning asynchronous DOM extraction, mathematical deduplication, WASM compilation, and temporal vector math. By systematically addressing these real-world complexities, the resulting architecture proves that an intelligent, highly responsive, and strictly private semantic index can exist entirely on the edge, redefining how individuals interact with their own digital history.

Works cited

1. Chrome Extension Manifest V3 Explained - OpenReplay Blog, <https://blog.openreplay.com/chrome-extension-manifest-v3/>
2. Chrome Manifest V3 Migration Guide - onHover, <https://onhover.in/blog/chrome-extension-manifest-v3-guide>
3. Use WebSockets in service workers | Chrome Extensions, https://developer.chrome.com/docs/extensions/how-to/web-platform/websocket_s
4. Offscreen Documents in Manifest V3 | Blog - Chrome for Developers, <https://developer.chrome.com/blog/Offscreen-Documents-in-Manifest-v3>

5. Building a local-first scraper extension that uses on-device ML to fix broken selectors. : r/chrome_extensions - Reddit,
https://www.reddit.com/r/chrome_extensions/comments/1to9x62/building_a_local_first_scraper_extension_that_uses/
6. PaneTrans — drag-select region translation + OCR on video/canvas, built on Transformers.js (local by default) - Hugging Face Forums,
<https://discuss.huggingface.co/t/panetrans-drag-select-region-translation-ocr-on-video-canvas-built-on-transformers-js-local-by-default/176929>
7. chrome.offscreen | API - Chrome for Developers,
<https://developer.chrome.com/docs/extensions/reference/api/offscreen>
8. How to Create Offscreen Documents in Chrome Extensions: A Complete Guide,
<https://dev.to/tearthian/how-to-create-offscreen-documents-in-chrome-extensions-a-complete-guide-3ke2>
9. How to Use Transformers.js in a Chrome Extension - Hugging Face,
<https://huggingface.co/blog/transformersjs-chrome-extension>
10. Message passing in Chrome extension (2024) - Victor on Software,
<https://victoronsoftware.com/posts/message-passing-in-chrome-extension/>
11. GitHub - kessler/gemma-gem: Gemma Gem runs Google's Gemma 4 model entirely on-device via WebGPU - Daily.dev,
<https://daily.dev/posts/github---kessler-gemma-gem-gemma-gem-runs-google-s-gemma-4-model-entirely-on-device-via-webgpu-no-bpv3ekj3e>
12. Transformers.js + ONNX Runtime WebGPU in Chrome extension | by Wei Lu | Medium,
<https://medium.com/@GenerationAI/transformers-js-onnx-runtime-webgpu-in-chrome-extension-13b563933ca9>
13. Message passing - Chrome for Developers,
<https://developer.chrome.com/docs/extensions/develop/concepts/messaging>
14. Content scripts || Message Passing || Web-accessible resources | by Naman Ahuja - Medium,
<https://medium.com/@palindromicnamed/content-scripts-message-passing-web-accessible-resources-d2e58b4b39d6>
15. Mozilla Readability Algorithm (Readability.js) explanation | WebcrawlerAPI Blog,
<https://webcrawlerapi.com/blog/mozilla-readability-algorithm-readabilityjs>
16. Web Scraping to Markdown: Extract and Clean Articles | theproductguy.in,
<https://theproductguy.in/blogs/web-scraping-to-markdown/>
17. Detecting YouTube SPA Navigation in a Chrome Extension Content Script,
<https://dev.to/ktg0215/detecting-youtube-spa-navigation-in-a-chrome-extension-content-script-2fi8>
18. webNavigation.onHistoryStateUpdated - Mozilla - MDN Web Docs,
<https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/webNavigation/onHistoryStateUpdated>
19. Prototype: Chrome extension to facilitate viewer context - API & Programming,
<https://community.fibery.io/t/prototype-chrome-extension-to-facilitate-viewer-context/5816>
20. Routing inside the chrome-extension · Issue #772 - GitHub,

- <https://github.com/crxjs/chrome-extension-tools/issues/772>
21. GoogleChrome/modern-web-guidance-src - GitHub,
<https://github.com/GoogleChrome/modern-web-guidance-src>
 22. I built an AI powered browser history search that runs entirely locally with no cloud and no accounts : r/SideProject - Reddit,
https://www.reddit.com/r/SideProject/comments/1s700ld/i_built_an_ai_powered_browser_history_search_that/
 23. A standalone version of the readability lib - GitHub,
<https://github.com/mozilla/readability>
 24. Readability.js,
<https://chromium.googlesource.com/external/github.com/mozilla/readability/+49bc5bfcc7704adc928dcd17e477fd36fd9f5679/README.md>
 25. @mozilla/readability - npm, <https://www.npmjs.com/package/@mozilla/readability>
 26. Near-Duplicate Detection with Simhash - Naman Kumar,
<https://naman.so/blog/simhash-web-crawl-caching>
 27. Detecting Near-Duplicates for Web Crawling - Google Research,
<https://research.google.com/pubs/archive/33026.pdf>
 28. vkandy/simhash-js - GitHub, <https://github.com/vkandy/simhash-js>
 29. What is SimHash? - DEV Community,
<https://dev.to/lovestaco/what-is-simhash-58m5>
 30. Simhash — algorithms documentation - Read the Docs,
<https://algonotes.readthedocs.io/en/latest/Simhash.html>
 31. SimHash - Wikipedia, <https://en.wikipedia.org/wiki/SimHash>
 32. sumonbis/NearDuplicateDetection: Near duplicate detection in a large collection of text using Locality Sensitive Hashing (LSH). - GitHub,
<https://github.com/sumonbis/NearDuplicateDetection>
 33. Speeding up SimHash by 10x using a bit hack - Dynatrace,
<https://www.dynatrace.com/news/blog/speeding-up-simhash-by-10x/>
 34. SimHash: A Quick Sketch of Set Similarity - Every Algorithm,
<https://every-algorithm.github.io/2023/12/20/simhash.html>
 35. Running Transformers.js inside a Chrome extension (Manifest V3): a practical patch | by Vprprudhvi | Medium,
<https://medium.com/@vprprudhvi/running-transformers-js-inside-a-chrome-extension-manifest-v3-a-practical-patch-d7ce4d6a0eac>
 36. Browser-Native Agents & LLMs: Run AI in Your Browser (2026) - Wow Data Science,
<https://wowdata.science/browser-native-agents-llms-in-browser-ai-guide-2026/>
 37. Chrome Extension Manifest V3 Migration: A New Era of Chrome Extensions Begin!, <https://binaryfolks.com/blog/chrome-extension-manifest-v3-migration/>
 38. Databases in the Browser - Parker Jones Dev Blog,
<https://parkerjones.dev/posts/databases-in-the-browser/>
 39. I built an on-device semantic search extension, basically a smarter Ctrl+F. It understands what you mean, not just what you type. : r/chrome_extensions - Reddit,
https://www.reddit.com/r/chrome_extensions/comments/1txjeg9/i_built_an_ondev

- [ice_semantic_search_extension/](#)
40. What is PGLite, <https://pglite.dev/docs/about>
 41. PGLite | Electric - ElectricSQL, <https://electric.ax/sync/pglite>
 42. electric-sql/pglite: Embeddable Postgres with real-time, reactive bindings. - GitHub, <https://github.com/electric-sql/pglite>
 43. PGLite: Run PostgreSQL in the Browser with WebAssembly - Noqta - AI Agents from \$5, <https://noqta.tn/en/tutorials/pglite-postgres-wasm-browser-local-first-2026>
 44. Site RAG - Chrome Web Store - Google, <https://chromewebstore.google.com/detail/site-rag/flahfjkiodmmchhbdieaihdcpmbofnak>
 45. PGLite API, <https://pglite.dev/docs/api>
 46. RAG Is Blind to Time — I Built a Temporal Layer to Fix It in Production, <https://towardsdatascience.com/rag-is-blind-to-time-i-built-a-temporal-layer-to-fix-it-in-production/>
 47. Temporal Grounding in RAG via Vector-Space Gating Instead of Post-Retrieval Filtering - Reddit, https://www.reddit.com/r/Rag/comments/1qp6ihw/temporal_grounding_in_rag_via_vectorspace_gating/
 48. Temporal RAG: Handling Time-Sensitive Information in Knowledge Bases - ChatNexus, <https://articles.chatnexus.io/knowledge-base/temporal-rag-handling-time-sensitive-information-i/>
 49. algotopian/chromeclaw: Your own personal AI assistant in Chrome/Firefox. - GitHub, <https://github.com/algotopian/chromeclaw>
 50. GitHub - marswangyang/personal-ai-memory: A local-first Chrome extension that passively captures ChatGPT, Gemini, Claude, Grok, Perplexity conversations into a private memory graph. Features in-browser Hybrid RAG (Vector + BM25), semantic search, and 100% privacy via WebAssembly and IndexedDB. No servers, no API keys., <https://github.com/marswangyang/personal-ai-memory>
 51. RAG Evaluation Methods: Separate Retrieval from Generation - Kili Technology, <https://kili-technology.com/blog/rag-evaluation-guide-measuring-retrieval-and-generation-as-separate-problems>
 52. RAG Evaluation: A Data Pipeline Performance Framework - Unstructured, <https://unstructured.io/insights/rag-evaluation-a-data-pipeline-performance-framework>
 53. [2604.18234] Evaluating Multi-Hop Reasoning in RAG Systems: A Comparison of LLM-Based Retriever Evaluation Strategies - arXiv, <https://arxiv.org/abs/2604.18234>
 54. MultiHop-RAG: Benchmarking Retrieval-Augmented Generation for Multi-Hop Queries - OpenReview, <https://openreview.net/pdf/43ab88c33fb4e13c3e723aaf46617f8e731d97c9.pdf>
 55. Self-hosted Lecture-to-Quiz: Local LLM MCQ Generation with Deterministic Quality Control, <https://arxiv.org/html/2603.08729v1>
 56. dbxcarta: Neo4j Semantic Layer for Databricks Unity Catalog - GitHub,

- <https://github.com/neo4j-partners/dbxcarta>
57. Repository for "MultiHop-RAG: A Dataset for Evaluating Retrieval-Augmented Generation Across Documents" (COLM 2024) - GitHub,
<https://github.com/yixuantt/MultiHop-RAG/>
 58. Best 9 RAG Evaluation Tools of 2026 - Deepchecks,
<https://deepchecks.com/best-rag-evaluation-tools/>
 59. RAG Evaluation - Ragas, https://docs.ragas.io/en/stable/howtos/cli/rag_eval/
 60. Streamline RAG applications with intelligent metadata filtering using Amazon Bedrock,
<https://aws.amazon.com/blogs/machine-learning/streamline-rag-applications-with-intelligent-metadata-filtering-using-amazon-bedrock/>
 61. AllowList - Chrome Web Store,
<https://chromewebstore.google.com/detail/allowlist/dolnbfmkeinkbflknmjgkkmhbghdkf>
 62. Allow or block apps and extensions - Chrome Enterprise and Education Help,
<https://support.google.com/chrome/a/answer/6177431?hl=en>
 63. Content scripts | Chrome for Developers,
<https://developer.chrome.com/docs/extensions/develop/concepts/content-scripts>